

Elaborazione 2

Introduzione

Durante la prima iterazione abbiamo progettato ed implementato alcune delle funzionalità chiave del sistema. Nell'iterazione corrente, similmente, cercheremo di sviluppare i casi d'uso funzionalmente più di valore per il sistema o che rappresentano un rischio per lo sviluppo futuro di quest'ultimo. Su questa falsa riga sono stati scelti per lo sviluppo i seguenti casi d'uso:

- UC6a: Crea Routine
- UC6b: Attiva Routine
- UC7: Monitoraggio

Alcune delle funzionalità dei casi d'uso **UC6a/b** si basano su **UC7: Monitoraggio** per cui progetteremo quest'ultimo per primo.

UC7: Monitoraggio

Analisi

Il monitoraggio si baserà sulla ricezione di messaggi (**MsgRete**) dai dispositivi fisici. Si prevede la ricezione di messaggi di **CAMBIO_STATO**, che trasportino le informazioni relative allo stato funzionale dei dispositivi, o di messaggi di **PING** periodici, per tracciare lo stato di connessione. A tale scopo è utile inserire un attributo **lastSeen** che memorizzi l'ultimo contatto tra il sistema ed il dispositivo fisico.

Modello di dominio

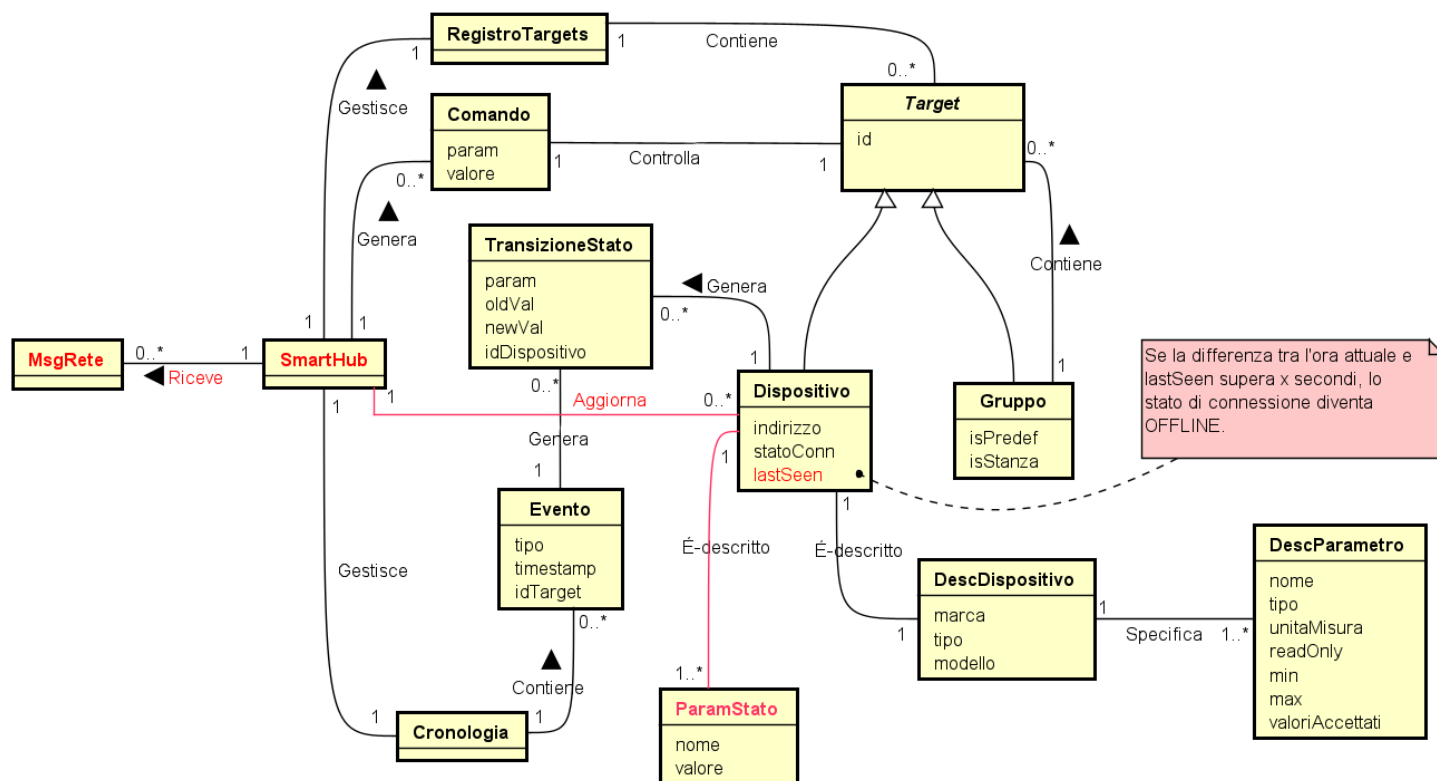
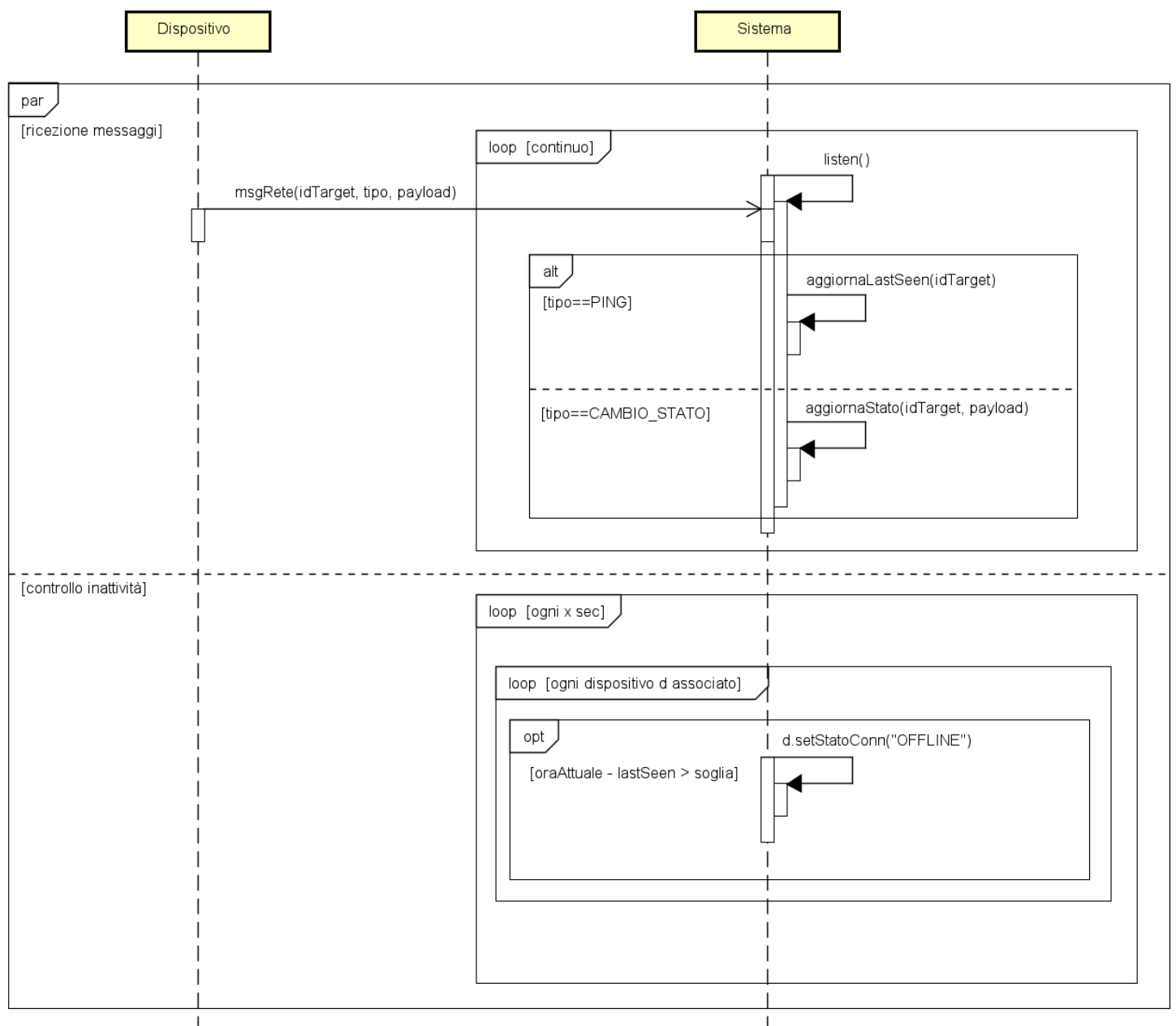


Diagramma sequenza di sistema

I messaggi ricevuti serviranno a mantenere la sincronizzazione tra i dispositivi ed il sistema e, in particolare, il sistema avrà anche il compito di controllare i periodi di inattività dei dispositivi al fine di determinarne lo stato di connessione.



Contratti

Contratto CO1: msgRete

- **Operazione:** msgRete(idTarget, tipo, payload)
- **Riferimenti:** UC7: Monitoraggio
- **Pre-condizioni:** Il sistema è attivo e funzionante.
- **Post-condizioni:**
 - **Caso A: Se tipo == PING
 - L'attributo lastSeen dell'istanza *d* di Dispositivo è stato aggiornato all'orario di sistema attuale.
 - Caso B: Se tipo == CAMBIO_STATO**
 - L'attributo valore dell'istanza *p* di ParamStato associata all'istanza *d* di Dispositivo è stato aggiornato con il dato contenuto in *payload*.
 - È stata creata una nuova istanza *t* di TransizioneStato .
 - È stata creata una nuova istanza *e* di Evento associata a *t*.
 - È stata formata un'associazione tra l'istanza *e* di Evento e l'istanza *c* di Cronologia .

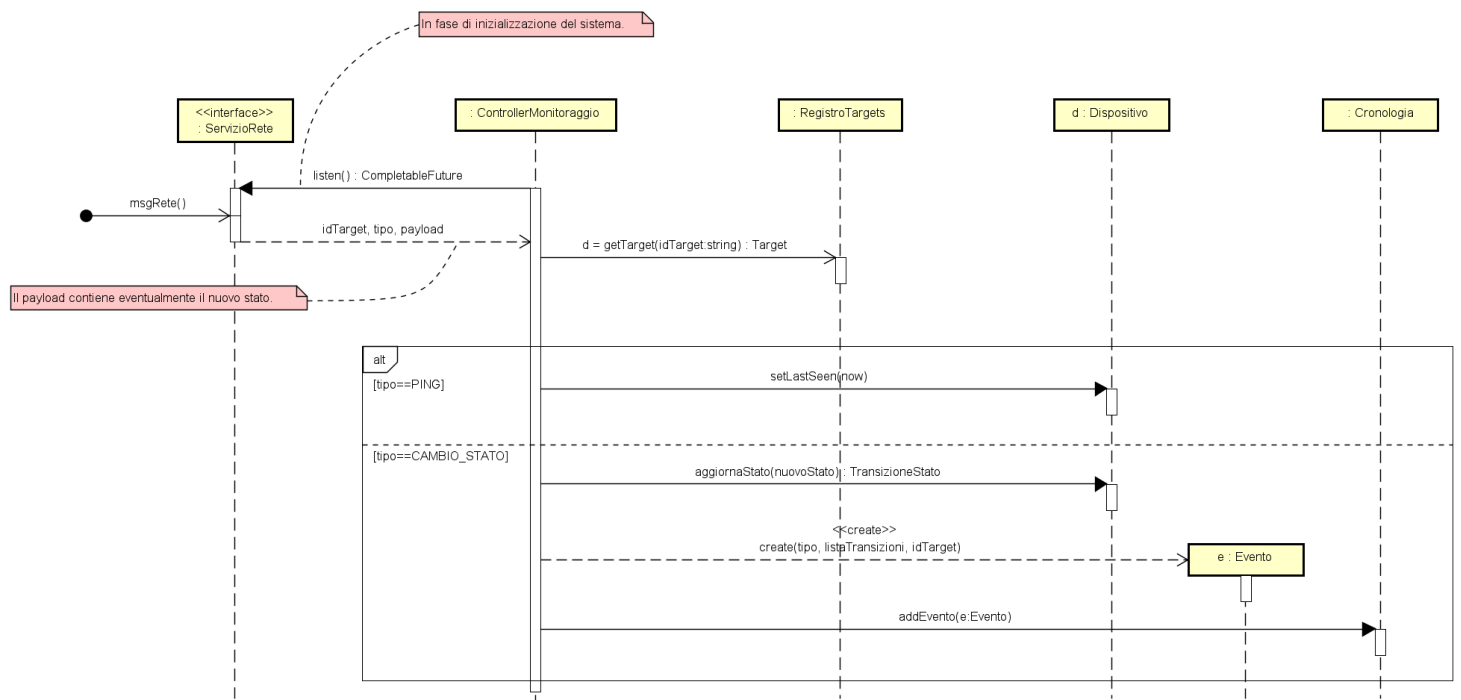
Contratto CO2: verificalnattivita

- **Operazione:** verificalnattivita(idTarget, tipo, payload)
- **Riferimenti:** UC7: Monitoraggio
- **Pre-condizioni:** Il sistema è attivo e funzionante.
- **Post-condizioni:** Per ogni istanza *d* di *Dispositivo* tale che $(oraAttuale - d.lastSeen) > SOGLIA_TIMEOUT$:
 - L'attributo *statoConn* di *d* è stato cambiato in *OFFLINE* .
 - È stata creata un'istanza *e* di *Evento* con tipo "DISCONNESSIONE".
 - L'istanza *e* è stata associata alla *Cronologia* .

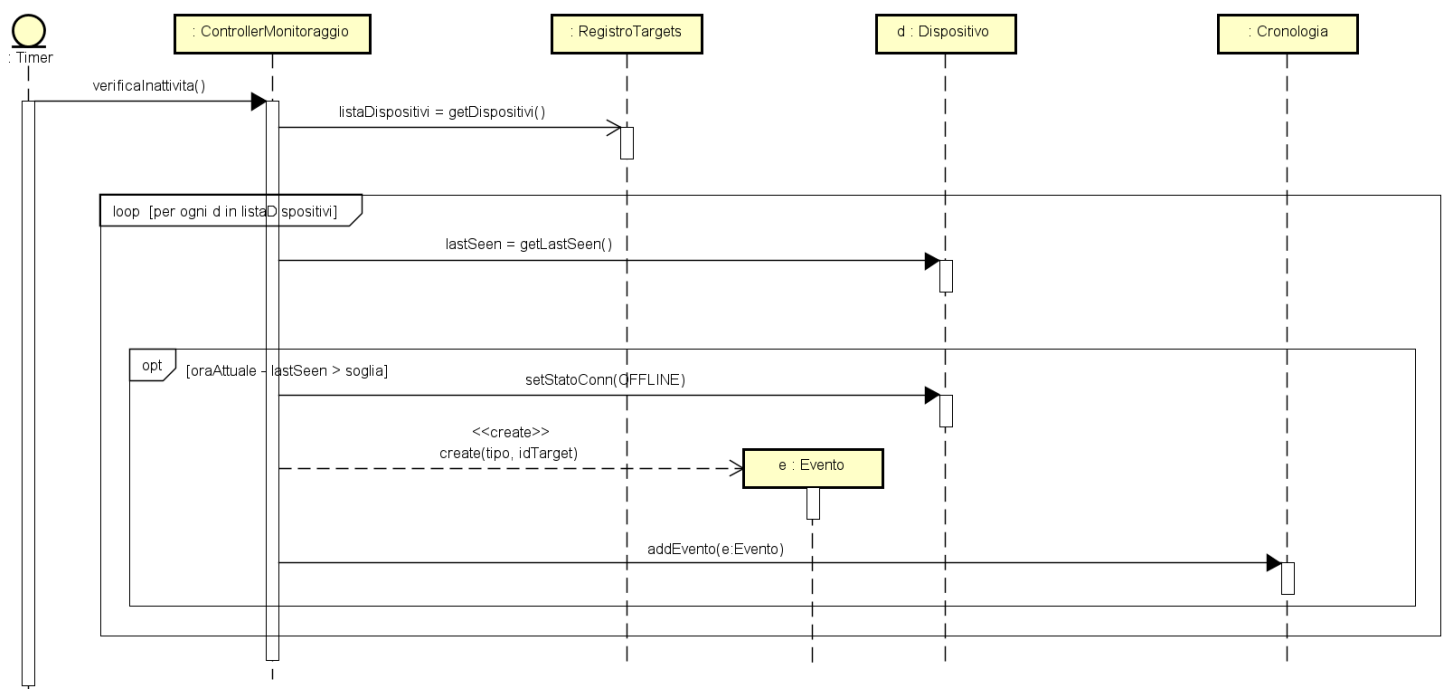
Progettazione

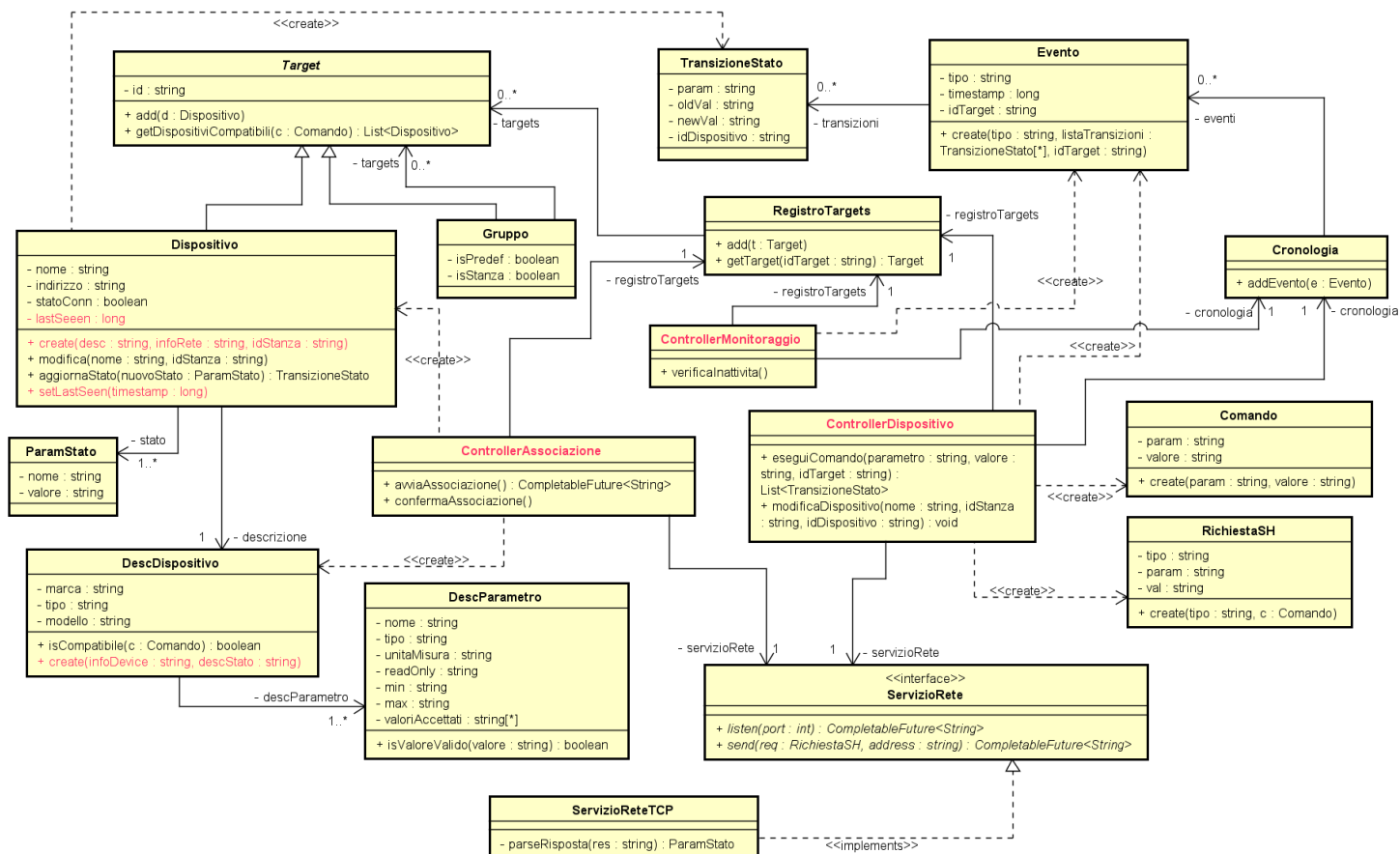
Diagrammi di sequenza

Op1: msgRete()



Op2: verificalnattivita()

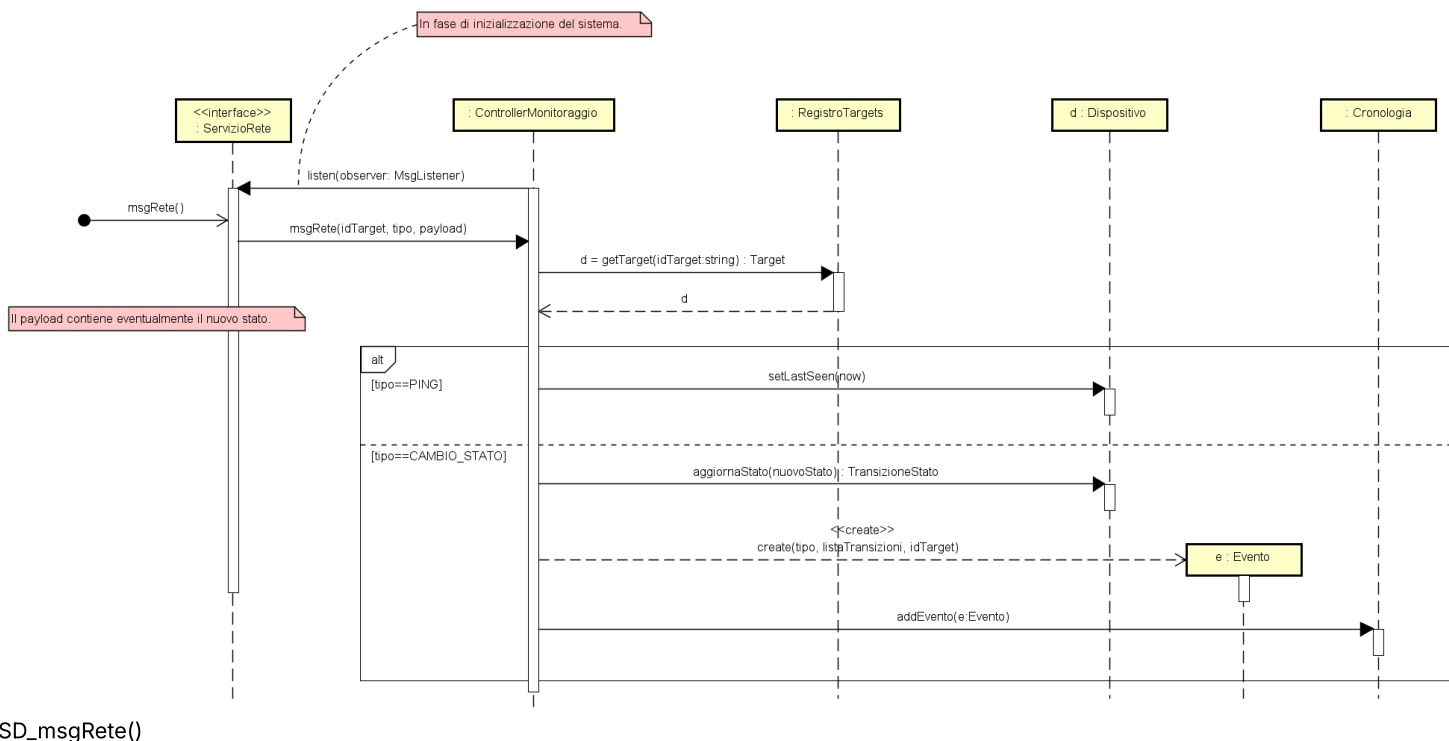




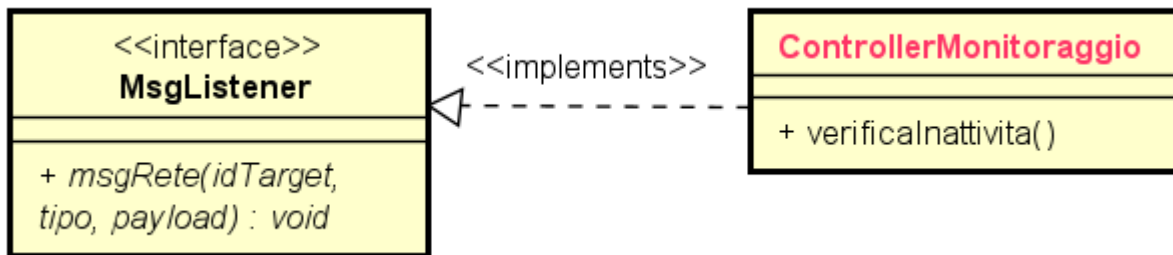
Per una visualizzazione ottimale vedi Progetto SmartHub.asta/Elaborazione2/UC7

Implementazione

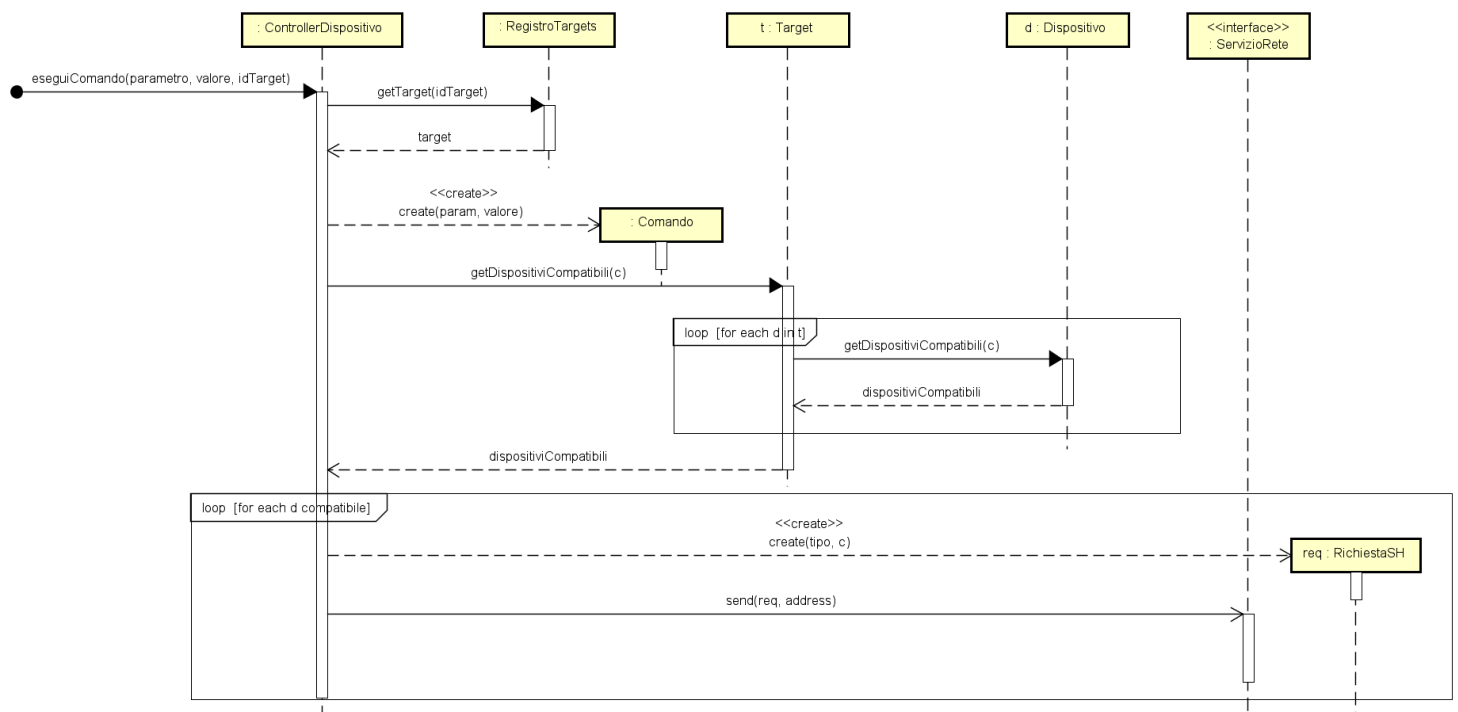
Durante l'implementazione l'utilizzo dei `CompletableFuture` per la gestione dei messaggi in ricezione dai dispositivi si è dimostrato inadeguato dunque si è optato per l'utilizzo di un'interfaccia *Observer* secondo il pattern omonimo. Di seguito il diagramma aggiornato:



In particolare `ControllerMonitoraggio` implementerà l'interfaccia `MsgListener` per gestione dei messaggi di rete.



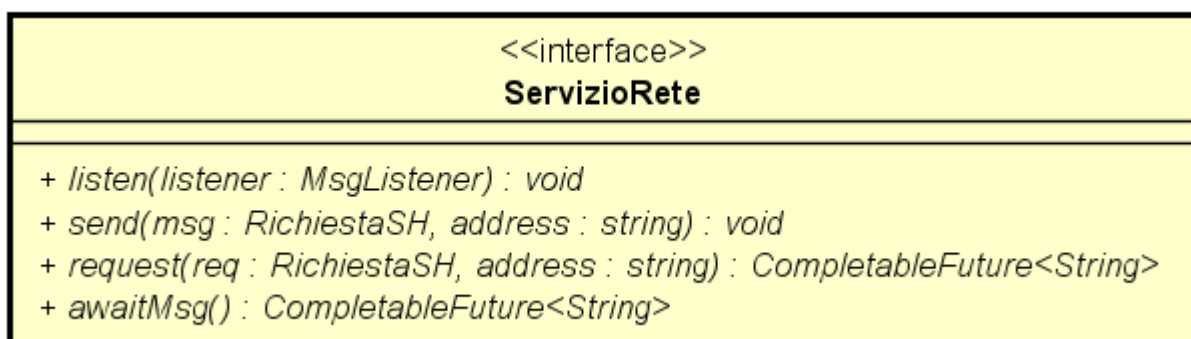
Inoltre, il metodo `eseguiComando` viene alleggerito delle responsabilità relative all'aggiornamento dello stato e della cronologia, le quali vengono interamente prese in carico dal sistema di monitoraggio. Ne risultano classi più coese e logiche dal punto di vista della separazione delle responsabilità. Di seguito il diagramma di sequenza aggiornato:



SD_eseguiComando()

Vorrei infine fare chiarezza sui metodi dell'interfaccia `ServizioRete`.

Sono stati utilizzati in diversi scenari metodi di ricezione/invio diversi. Talvolta è stata preferita una gestione tramite `CompletableFuture`, talvolta una gestione puramente asincrona (magari supportata dal sistema di monitoraggio). Si definiscono dunque i metodi `request()` e `awaitMsg()`, che si prestano più ad interazioni singole e vengono dunque utilizzati durante l'associazione dei dispositivi. Ed i metodi `listen()` e `send()` che sono rispettivamente pensati per una ricezione continua (Monitoraggio) e per un'invio di messaggi secondo il paradigma *fireNforget* (Comanda Dispositivo).

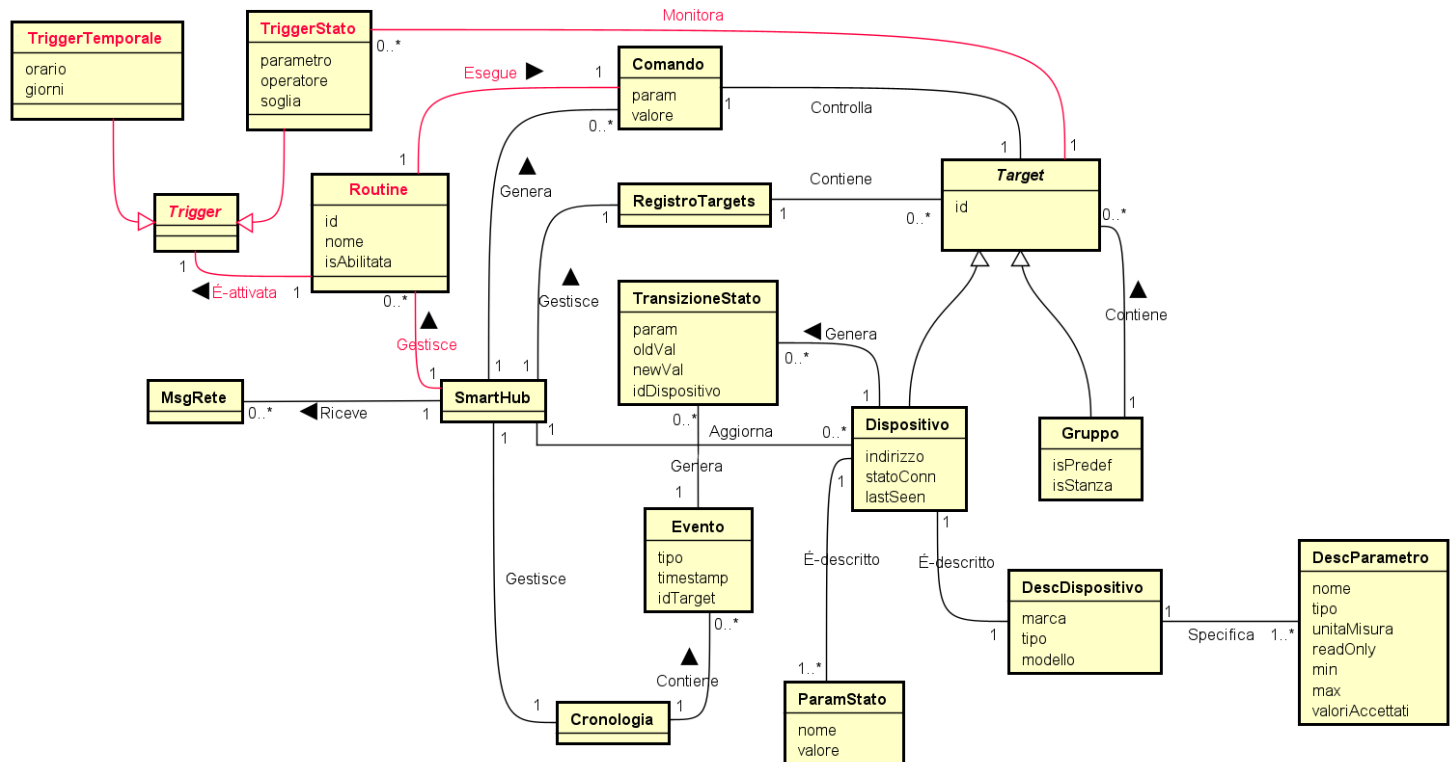


UC6: Crea Routine / Attiva Routine

Analisi

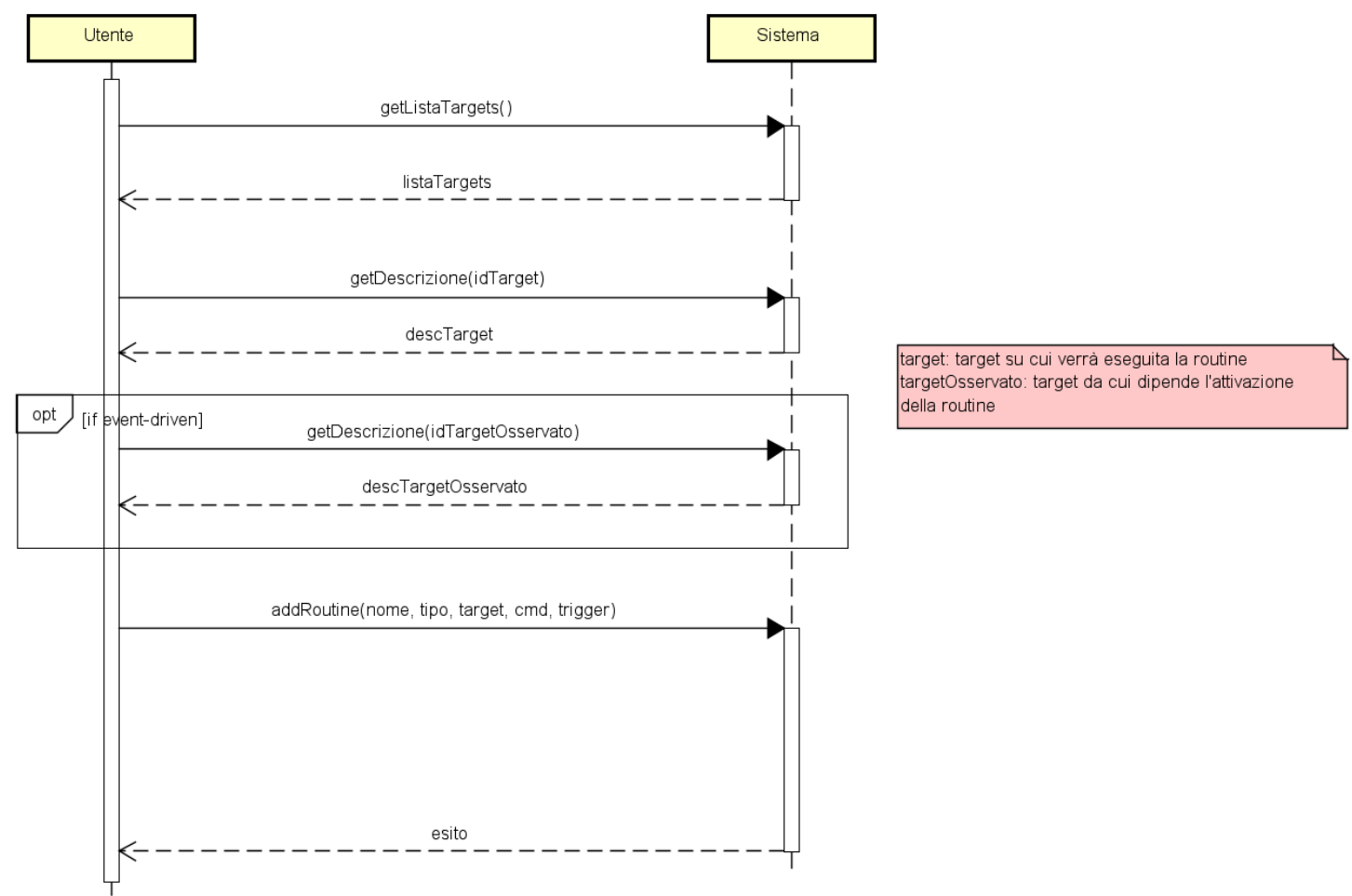
Una routine deve essere in grado di eseguire comandi su target condizionalmente all'attivazione di trigger definiti dall'utente.

Modello di Dominio

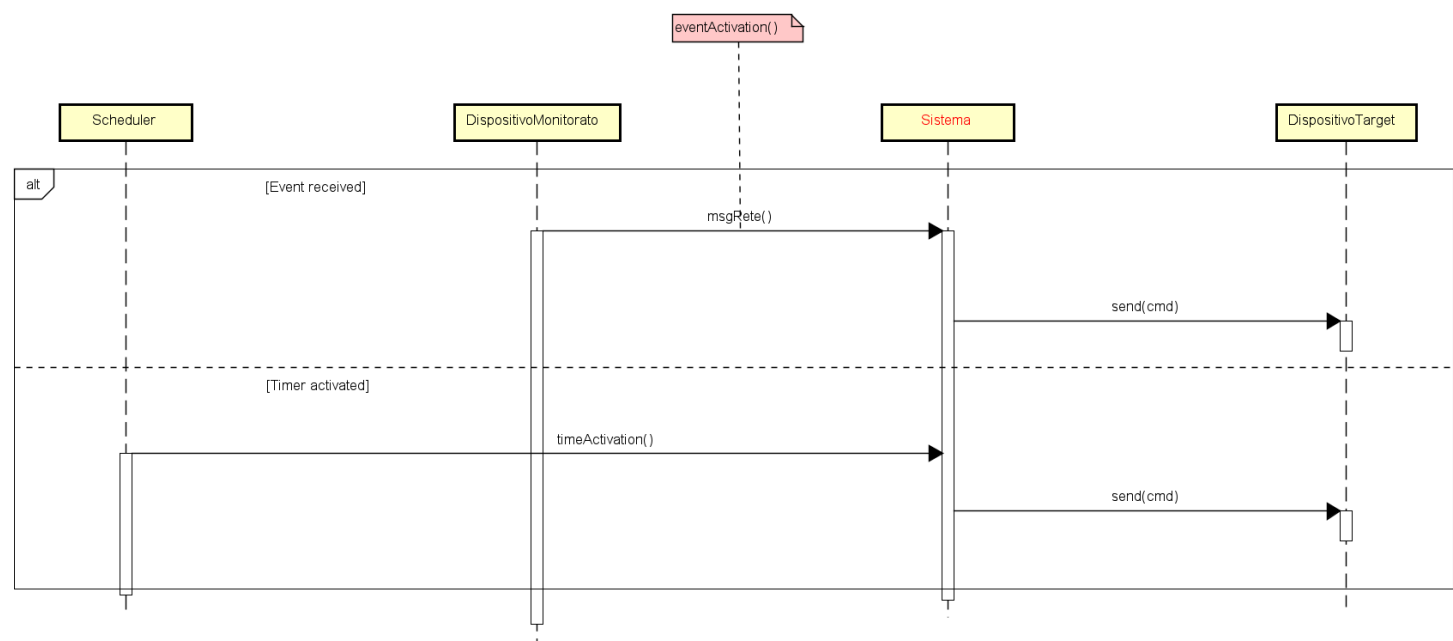


TriggerStato e *TriggerTemporale* devono necessariamente avere condizioni e modalità di attivazione differenti.

Diagrammi di sequenza di sistema



UC6a: Crea Routine



UC6b: Attiva Routine

Contratti UC6a: Crea Routine

Contratto C01: getListaTargets

- **Operazione:** getListaTargets()
- **Riferimenti:** UC6a: Crea Routine
- **Pre-condizioni:** Il sistema è attivo e il registro dei dispositivi contiene almeno un Target validamente associato.
- **Post-condizioni:** Lo stato del sistema rimane inalterato.

Contratto CO2: getDescrizione

- **Operazione:** getDescrizione()
- **Riferimenti:** UC6a: Crea Routine
- **Pre-condizioni:** Esiste nel sistema un `Target` il cui identificativo corrisponde a `idTarget`.
- **Post-condizioni:** Lo stato del sistema rimane inalterato.

Contratto CO3: addRoutine

- **Operazione:** addRoutine(nome, tipo, target, cmd, trigger)
- **Riferimenti:** UC6a: Crea Routine
- **Pre-condizioni:**
 - Il dispositivo identificato da `target` ed eventualmente il dispositivo da monitorare menzionato in `trigger` esistono nel sistema.
 - L'azione descritta in `cmd` è formalmente valida e supportata dal dispositivo `target`.
 - I parametri temporali o condizionali forniti in `trigger` rispettano il formato previsto.
- **Post-condizioni:**
 - È stata creata una nuova istanza `c` della classe `Comando`, inizializzata con i dati di `cmd`.
 - È stata creata una nuova istanza `tr` della classe `TriggerTemporale` o della classe `TriggerStato` ed è stata inizializzata con i dati contenuti nel parametro `trigger` (contiene eventualmente anche il target osservato).
 - È stata creata una nuova istanza `r` della classe `Routine`
 - L'attributo `nome` dell'istanza `r` è stato inizializzato con il parametro `nome`.
 - L'attributo `isAbilitata` dell'istanza `r` è stato impostato a `true`.
 - L'attributo `tipo` di `r` è stato inizializzato al parametro `tipo`.
 - L'istanza `c` di `Comando` è stata associata a `r`
 - L'istanza `t` di `Target` è stata associata a `r`
 - L'istanza `tr` di `Trigger` è stata associata a `r`
 - L'istanza `r` di `Routine` è stata memorizzata nel `MotoreRoutine`
 - Se `tipo == 'TIME'` il `MotoreRoutine` ha schedulato l'evento relativo alla routine.

Contratti UC6b: Attiva Routine

Contratto CO1: eventActivation (Integrazione dell'operazione `msgRete` descritta in UC7)

- **Operazione:** msgRete(idTarget, tipo, payload)
- **Riferimenti:** UC7: Monitoraggio, UC6b: Attiva Routine
- **Pre-condizioni:** Il sistema è attivo e funzionante, e `idTarget` corrisponde a un dispositivo validamente registrato.
- **Post-condizioni:**
 - ****Caso A:** Se `tipo == PING`
 - L'attributo `lastSeen` dell'istanza `d` di `Dispositivo` è stato aggiornato all'orario di sistema attuale.
 - **Caso B:** Se `tipo == CAMBIO_STATO`
 - L'attributo `valore` dell'istanza `p` di `ParamStato` associata all'istanza `d` di `Dispositivo` è stato aggiornato con il dato contenuto in `payload`.
 - È stata creata una nuova istanza `t` di `TransizioneStato`.
 - È stata creata una nuova istanza `e` di `Evento` associata a `t`.
 - È stata formata un'associazione tra l'istanza `e` di `Evento` e l'istanza `c` di `Cronologia`.
- **eventActivation()**
 - Se esiste almeno un'istanza `r` della classe `Routine` tale che:
 - `r.isAbilitata() == true`
 - il `TriggerStato` `tr` associato a `r` è associato ad un `Target` avente `id == idTarget`
 - `tr` valuta come soddisfatta la condizione di attivazione
 - Allora il `Comando` associato a `r` sarà stato eseguito

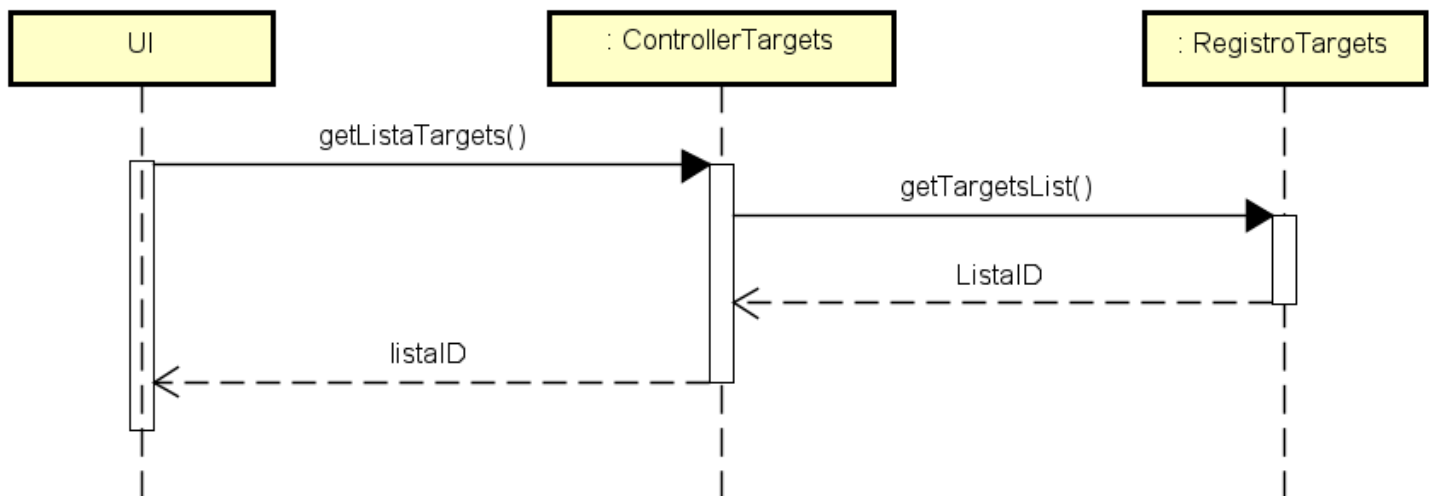
Contratto CO2: timeActivation

- **Operazione:** timeActivation()
- **Riferimenti:** UC6b: Attiva Routine
- **Pre-condizioni:** Il tempo di attesa calcolato alla creazione della routine `r` è scaduto.
- **Post-condizioni:**
 - Se l'istanza `r` ha ancora l'attributo `isAbilitata == true`:
 - Il Comando associato a `r` sarà stato eseguito
 - Il prossimo evento previsto dalla Routine `r` sarà stato schedulato

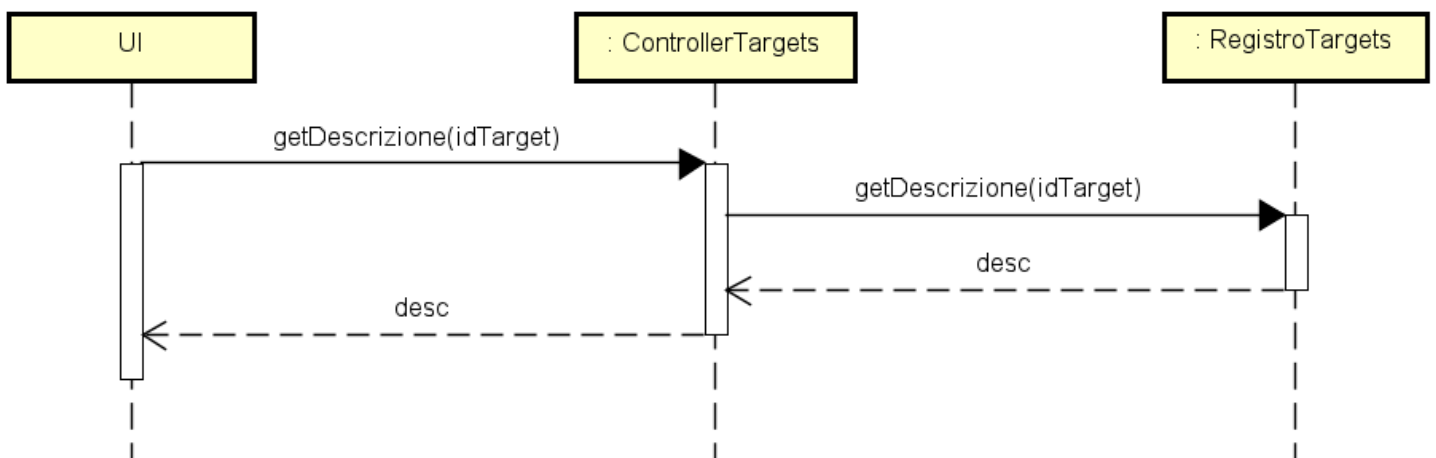
Progettazione

Diagrammi di sequenza

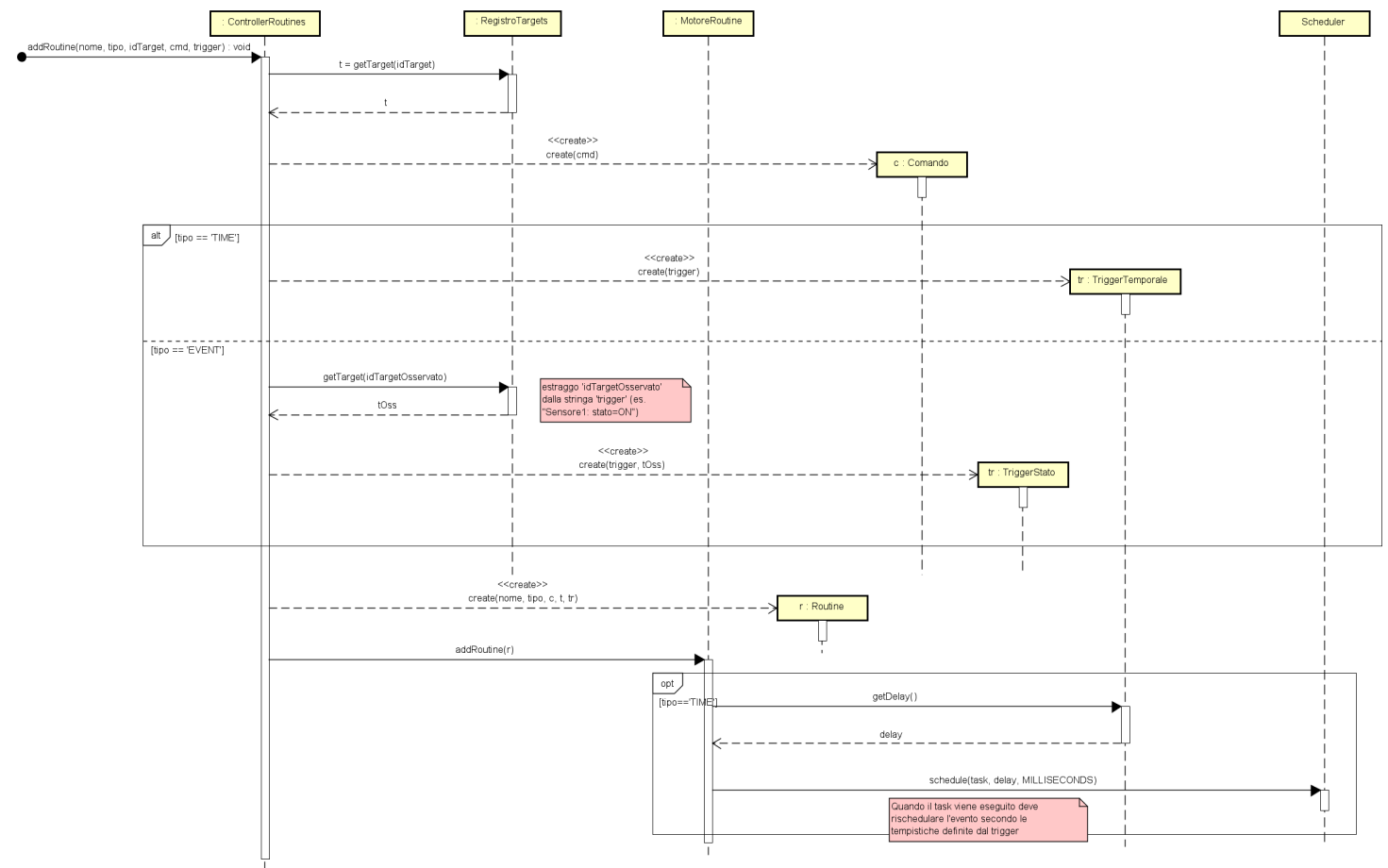
uc6a_sd_getListaTargets()



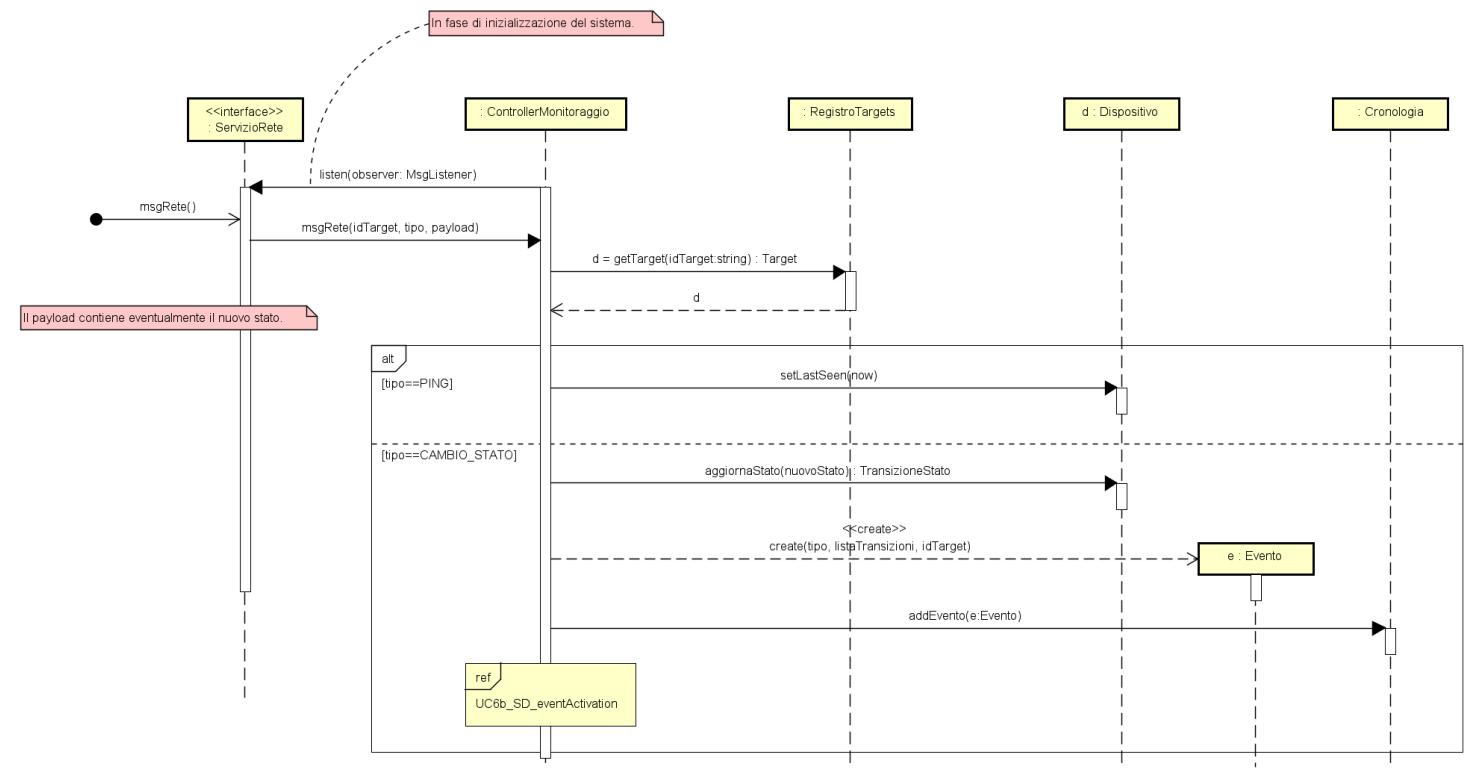
uc6a_sd_getDescrizione()



uc6a_sd_addRoutine()

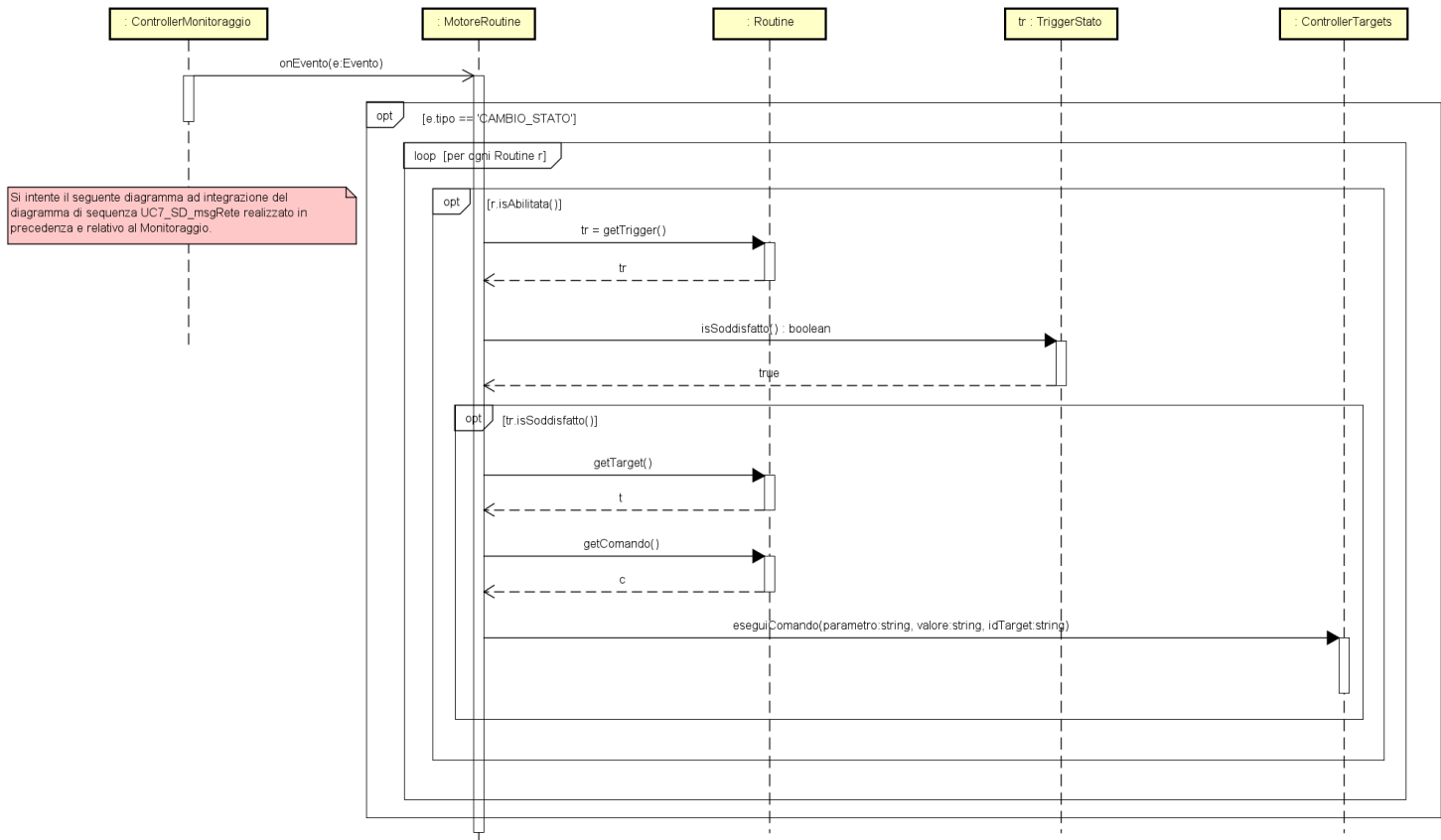


uc6b_sd_msgRete() → eventActivation()



msgRete()

Il diagramma di sequenza dell'operazione **msgRete()** definito durante la progettazione del caso d'uso **UC7: Monitoraggio** viene completato dal riferimento a **UC6_SD_eventActivation()** definito di seguito.



eventActivation()

uc6b_sd_timeActivation()

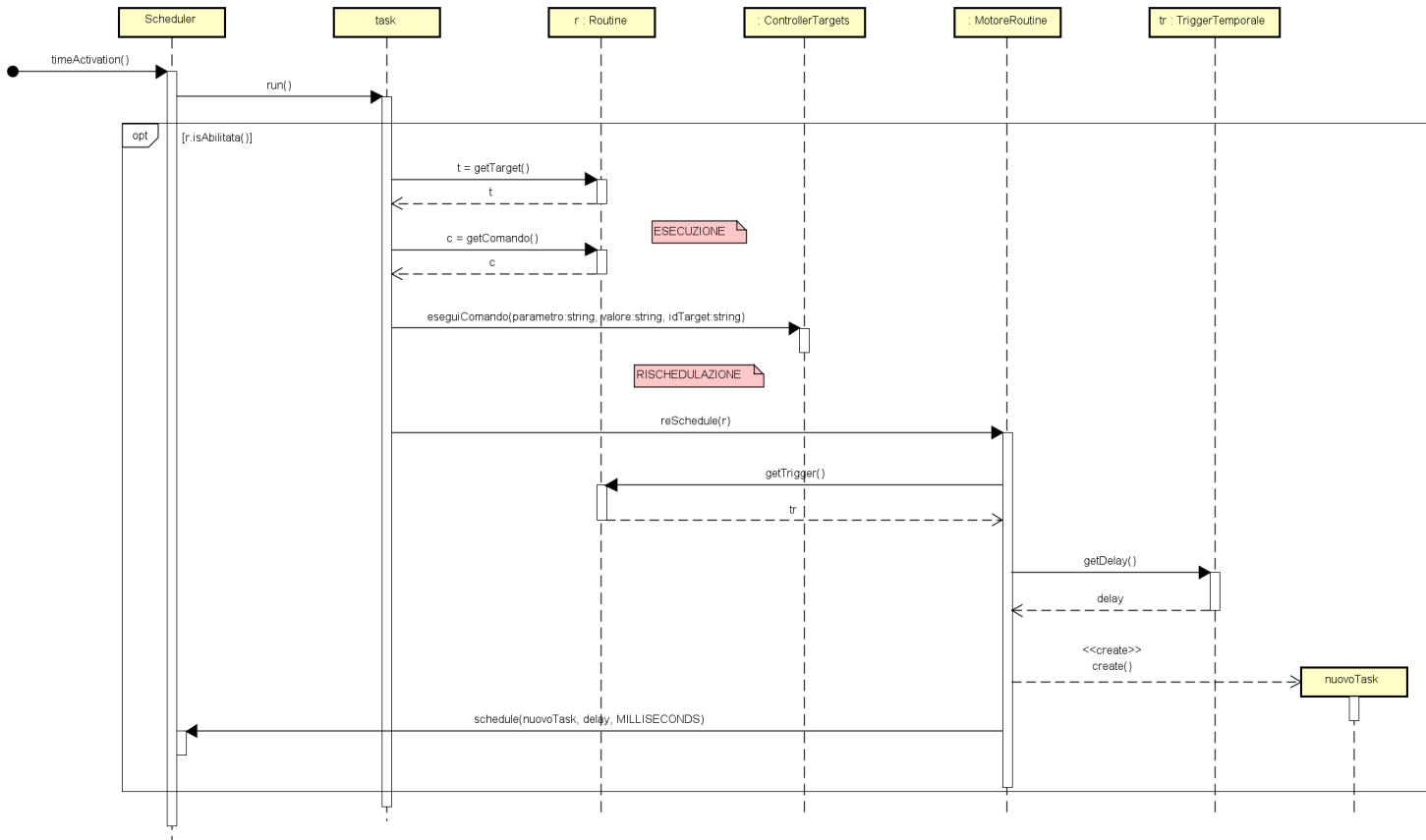


Diagramma delle classi

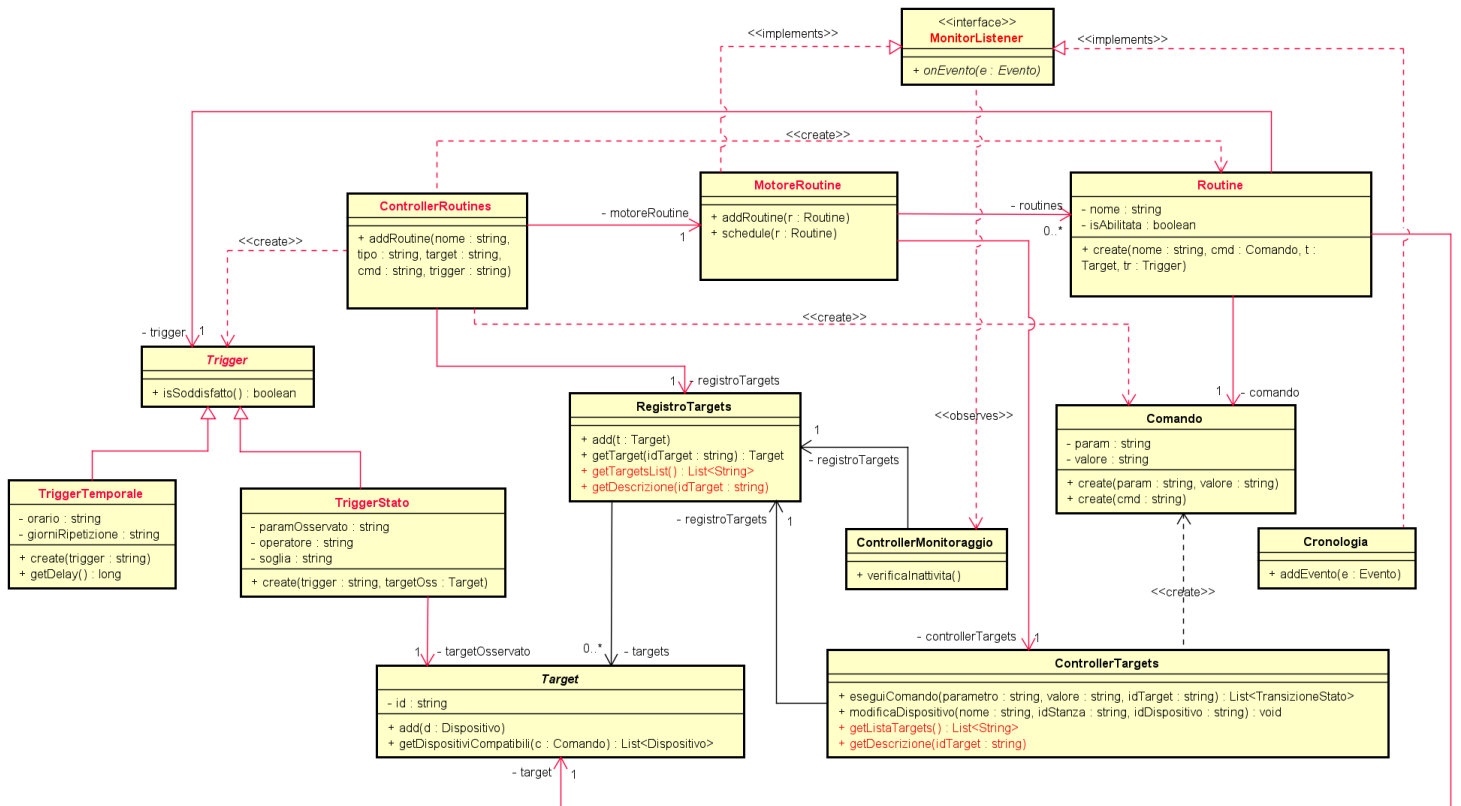


Diagramma parziale delle classi relativo al caso d'uso UC6a/b: Crea/Attiva Routine. Per una visualizzazione ottimale o per il diagramma delle classi completo riferirsi al progetto Smarthub.asta relativo all'iterazione corrente.

Come si evince dal diagramma delle classi, la classe `ControllerDispositivi` è stata rinominata in `ControllerTargets`, in quanto si è ritenuto il nome più appropriato alle responsabilità della classe. Inoltre è stata inserita un'interfaccia `MonitorListener` per l'osservazione degli eventi rilevati dal `ControllerMonitoraggio: MotoreRoutine` e `Cronologia` implementano tale interfaccia.

Implementazione

Nel corso dell'iterazione le seguenti classi sono state implementate:

- Trigger, TriggerStato e TriggerTemporale
- Routine
- L'interfaccia MonitorListener
- L'interfaccia MsgListener
- ControllerMonitoraggio
- ControllerRoutines

Inoltre le seguenti classi hanno subito delle modifiche:

- RegistroTargets
- ControllerTargets
- Comando
- Dispositivo
- Cronologia
- L'interfaccia ServizioRete e ServizioReteTCP

Infine sono stati modificati le classi che simulano i dispositivi fisici e l'interfaccia provvisoria del sistema.